Gimbal Task & Auto-Aim Receive

Sentry Board Control · ■■■■ & ■■■■■■■■

■■

1. Gimbal_Task — ■■■■■■

1.1 ■■■■■■■■

1.2 S1=UP — ■■■■

1.3 S1=MID — ■■■■

1.4 S1=DOWN — ■■■■

2. GimbalAimReceive_Task — ■■ CAN ■■

2.1 CAN ■■■

2.2 ■■■■■■■

1. Gimbal_Task — ■■■■■■

■■■■■■■■ FreeRTOS ■■ 1ms ■■■■■■■■■■■■ (S1/S2) ■■■■■■■

S1	S2	■■	■■
UP	MID	■■	■■■■, ■■■■■
UP	UP	■■■■	■■■■ Yaw/Pitch
UP	DOWN	■■	■■■ offset, ■■■■■
MID	MID	■■	■■■■
MID	DOWN	■■■■	■■■■ + ■■■■■■■
MID	UP	■■+■■	■■■■, ■■■■■

1.1 ■■■■

```
void Gimbal_Task(void *argument)

{

    Gimbal_Init();

    AutoControl_Init();


    float autoAimYawSetAngle = yaw_offset;

    float autoAimPitchSetAngle = pitch_offset;

    float autoAimPitchGetAngle = pitch_offset, autoAimYawGetAngle = yaw_offset;


    static uint8_t auto_aim_active = 0;

    static uint32_t last_auto_aim_rx_seq = 0;

    static uint32_t auto_aim_lose_start_time = 0;

    static uint8_t auto_aim_has_received = 0;

    static uint8_t auto_aim_scan_seeded = 0;

    static uint8_t auto_aim_branch_active = 0;


    static uint8_t stable_auto_aim_inited = 0;

    static uint8_t stable_auto_aim_mode_active = 0;


    float pitch_current;

    float rc_target_yaw = yaw_offset;


    if (stable_auto_aim_inited == 0U)

    {

        AutoAimStable_Init(&stable_auto_aim);

        stable_auto_aim_inited = 1U;

    }
```

```c
int16_t yaw_rc_init = 0;

for (;;)
{
    uint8_t rc_auto_aim_branch_now =
        ((RC_Ctl.rc.s1 == RC_SW_MID) && (RC_Ctl.rc.s2 == RC_SW_DOWN)) ? 1U : 0U;

    M6020_GetBasicData(&yaw_gimbal_motor);
    yaw_gimbal_motor.basic_data.ref_angle =
        ((yaw_gimbal_motor.basic_data.ecd / 8192.0f) * 360.0f) - 180.0f;

    float yaw_ref_angel = yaw_gimbal_motor.basic_data.ref_angle;

    motor1_init(&DM_4310);

    if ((rc_auto_aim_branch_now == 0U) && (auto_aim_branch_active != 0U))
    {
        auto_aim_branch_active = 0U;
        auto_aim_has_received = 0U;
        auto_aim_active = 0U;
        auto_aim_scan_seeded = 0U;
        auto_aim_lose_start_time = 0U;
        last_auto_aim_rx_seq = auto_aim_rx_seq;
    }

    if (!yaw_rc_init) {
        yaw_ref_angel = rc_target_yaw;
        yaw_rc_init = 1;
    }

    /* ---- PC ■■■■ (~200Hz) ---- */
    static uint32_t last_pc_tx = 0;
    if (HAL_GetTick() - last_pc_tx >= 5)
    {
        last_pc_tx = HAL_GetTick();
        CAN_cmd_PC(yaw_gimbal_motor.basic_data.ecd, DM_4310.para.pos);
    }
```

```
        /* ============ S1 == UP ============ */
        if (RC_Ctl.rc.s1 == RC_SW_UP)
        {
            if (RC_Ctl.rc.s2 == RC_SW_MID)
            {
                Shoot_TriggerAndFrictionWheelControl(0, 0);
                pitch_spevel_control(pitch_offset);
            }
            else if (RC_Ctl.rc.s2 == RC_SW_UP)
            {
                // Shoot_TriggerAndFrictionWheelControl(-3000, 0);

                rc_target_yaw += (RC_Ctl.rc.ch0 * yaw_sensitivy);
                pitch_current = DM_4310.para.pos;
                rc_target_pitch = (RC_Ctl.rc.ch1 + 78.0f) * pitch_sensitivy + pitch_current;
                rc_target_pitch = Float_Limit(rc_target_pitch, PITCH_MIN, PITCH_MAX);
                Yaw_Motor_Control(rc_target_yaw);
                pitch_spevel_control(rc_target_pitch);
            }
            else if (RC_Ctl.rc.s2 == RC_SW_DOWN)
            {
                pitch_spevel_control(pitch_offset);
                Yaw_Motor_Control(yaw_offset);
                // Shoot_TriggerAndFrictionWheelControl(-4800, 2000);
            }
        }
        /* ============ S1 == MID ============ */
        else if (RC_Ctl.rc.s1 == RC_SW_MID)
        {
            if (RC_Ctl.rc.s2 == RC_SW_MID)
            {
                pitch_spevel_control(pitch_offset);
                Yaw_Motor_Control(yaw_offset);
            }
            else if (RC_Ctl.rc.s2 == RC_SW_DOWN)
```

```
{
    /* ---------- ■■■■ ---------- */
    uint32_t now_tick = HAL_GetTick();
    uint8_t auto_aim_state = GetAutoAIM();

    // ■■■■■
    uint8_t auto_aim_timeout =
        (auto_aim_active != 0U) &&
        (auto_aim_last_rx_tick != 0U) &&
        ((now_tick - auto_aim_last_rx_tick) > AUTO_AIM_TIMEOUT_MS);

    if (auto_aim_branch_active == 0U)
    {
        auto_aim_branch_active = 1U;
        auto_aim_has_received = 0U;
        auto_aim_active = 0U;
        auto_aim_scan_seeded = 0U;
        auto_aim_lose_start_time = 0U;
        last_auto_aim_rx_seq = auto_aim_rx_seq;
    }

    // ■■■■■■■■■■■■■■■■■
    if ((auto_aim_state == AUTO_AIM_GET) && (auto_aim_timeout == 0U))
    {
        auto_aim_has_received = 1;
        auto_aim_active = 1;
        auto_aim_lose_start_time = 0;
        auto_aim_scan_seeded = 0U;

        // ■■■■■■■■■■■■■■■■
        if (last_auto_aim_rx_seq != auto_aim_rx_seq)
        {
            last_auto_aim_rx_seq = auto_aim_rx_seq;

            autoAimPitchGetAngle = DM_4310.para.pos;
            autoAimYawGetAngle = yaw_gimbal_motor.basic_data.ref_angle;

            autoAimYawSetAngle = autoAimYawGetAngle
```

```
                + (auto_aim.datafloat.d_yaw);
            autoAimPitchSetAngle = autoAimPitchGetAngle
                + (auto_aim.datafloat.d_pitch + 2);

            autoAimYawSetAngle = Float_Limit(
                autoAimYawSetAngle, YAW_MIN_ANGLE_DEG, YAW_MAX_ANGLE_DEG);
            autoAimPitchSetAngle = Float_Limit(
                autoAimPitchSetAngle, PITCH_MIN, PITCH_MAX);
        }

        Yaw_Motor_Control(autoAimYawSetAngle);
        pitch_spevel_control(autoAimPitchSetAngle);
    }
    // ■■■■ / ■■■■■■ → ■■■■
    else if (auto_aim_has_received &&
        ((auto_aim_state == AUTO_AIM_LOSE) ||
         (auto_aim_timeout != 0U)))
    {
        uint32_t lose_duration;

        if (auto_aim_lose_start_time == 0)
        {
            auto_aim_lose_start_time = now_tick;
        }
        lose_duration = now_tick - auto_aim_lose_start_time;

        // ■■1: ■■ (■■■■■■■■■)
        if (lose_duration <= AUTOAIM_HOLD_MS)
        {
            Yaw_Motor_Control(autoAimYawSetAngle);
            pitch_spevel_control(autoAimPitchSetAngle);
        }
        // ■■2: ■■
        else if (lose_duration <= (AUTOAIM_HOLD_MS + AUTOAIM_SCAN_MS))
        {
            if (auto_aim_scan_seeded == 0U)
            {
```

```
                    yaw_set = autoAimYawSetAngle;
                    pitch_set = autoAimPitchSetAngle;
                    auto_aim_scan_seeded = 1U;
                }
                yaw_con();
                pitch_con();
            }
            // ■■3: ■■■■■■
            else
            {
                autoAimYawSetAngle = yaw_offset;
                autoAimPitchSetAngle = pitch_offset;
                Yaw_Motor_Control(yaw_offset);
                pitch_spevel_control(pitch_offset);

                auto_aim_active = 0;
                auto_aim_has_received = 0;
                auto_aim_scan_seeded = 0U;
                auto_aim_lose_start_time = 0;
                auto_aim.aim_signal = AUTO_AIM_LOSE;
                auto_aim.datafloat.d_yaw = 0.0f;
                auto_aim.datafloat.d_pitch = 0.0f;
                auto_aim.datafloat.last_d_yaw = 0.0f;
                auto_aim.datafloat.last_d_pitch = 0.0f;
            }
        }
        else if (auto_aim_has_received == 0)
        {
            auto_aim_active = 0;
            auto_aim_scan_seeded = 0U;
        }
    }
    else if (RC_Ctl.rc.s2 == RC_SW_UP)
    {
        pitch_spevel_control(pitch_offset);
```

```
                Yaw_Motor_Control(yaw_offset);
                Shoot_TriggerAndFrictionWheelControl(0, 2000);
            }
        }
        /* ============ S1 == DOWN ============ */
        else if (RC_Ctl.rc.s1 == RC_SW_DOWN)
        {
            if (stable_auto_aim_mode_active != 0U)
            {
                AutoAimStable_Reset(&stable_auto_aim);
                stable_auto_aim_mode_active = 0U;
            }
        }

        osDelay(1);
    }
}
```

2. GimbalAimReceive_Task — ■■ CAN ■■

■■■■■ CAN ■■■■■■ (■■■■■■) ■■■■■■■■■■ Yaw/Pitch
■■■■■■■■■■

```
    continue;

// ■■ CAN ■: Yaw / Pitch / Aim / Shoot
int16_t raw_yaw = (int16_t)(queueData.rx_data[0] << 8 | queueData.rx_data[1]);
int16_t raw_pitch = (int16_t)(queueData.rx_data[2] << 8 | queueData.rx_data[3]);
uint16_t raw_aim = (uint16_t)(queueData.rx_data[4] << 8 | queueData.rx_data[5]);
uint16_t raw_shoot = (uint16_t)(queueData.rx_data[6] << 8 | queueData.rx_data[7]);

// ■■■■■■■ (■■■■■■)
auto_aim.datafloat.last_d_yaw = auto_aim.datafloat.d_yaw;
auto_aim.datafloat.last_d_pitch = auto_aim.datafloat.d_pitch;

// ■■■■
if (raw_aim == 1)
{
    auto_aim.aim_signal = AUTO_AIM_GET;
    AutoAim_SendToYaw_baseBoard(1);
}
else
{
    auto_aim.aim_signal = AUTO_AIM_LOSE;
}

// ■■■■
if (raw_shoot == 1)
    auto_aim.shoot_signal = AUTO_AIM_SHOOT_TURN_ON;
else
    auto_aim.shoot_signal = AUTO_AIM_SHOOT_TURN_OFF;

// ■■■■ (deg), ■■ 100
auto_aim.datafloat.d_yaw = raw_yaw / 100.0f;
auto_aim.datafloat.d_pitch = raw_pitch / 100.0f;

// ■■■■■■: 0.6×■■ + 0.4×■■■
auto_aim.datafloat.d_yaw =
    auto_aim.datafloat.d_yaw * 0.6f +
    auto_aim.datafloat.last_d_yaw * 0.4f;
```

```
        auto_aim.datafloat.d_pitch =
            auto_aim.datafloat.d_pitch * 0.6f +
            auto_aim.datafloat.last_d_pitch * 0.4f;

        // ■■: ■■ 0.3° ■■
        if (fabsf(auto_aim.datafloat.d_yaw) < 0.3f)
            auto_aim.datafloat.d_yaw = 0.0f;

        if (fabsf(auto_aim.datafloat.d_pitch) < 0.3f)
            auto_aim.datafloat.d_pitch = 0.0f;

        // ■■■■■■■■ rx_seq ■■■■■■■
        auto_aim_rx_seq++;
        auto_aim_last_rx_tick = HAL_GetTick();
    }
}
```